

Ejercicio 4

Investigación REACT

¿Qué son los estados en React ?

El estado (*state*) es la información que un componente guarda y que puede cambiar con el tiempo (un contador, el texto de un input, si un menú está abierto o cerrado). Cuando el estado cambia, React vuelve a renderizar el componente para mostrar la interfaz actualizada. Sirve para crear interfaces dinámicas que responden a las acciones del usuario.

¿Cuáles son sus características y para qué sirve?

- ☆ Es propio de cada componente (local y privado).
- ☆ Se declara con el Hook `useState`, que devuelve el valor actual y una función para modificarlo.
- ☆ Nunca se modifica directamente: se debe usar la función setter (`setContador`), no asignar el valor.
- ☆ Cada vez que cambia, el componente se vuelve a renderizar.
- ☆ A diferencia de los props, el estado sí puede cambiar dentro del componente.
- ☆ Puede guardar cualquier tipo de dato: números, textos, booleanos, objetos o arrays.

Ejemplo de implementación:

```
1  import { useState } from "react";
2
3  function Contador() {
4    const [contador, setContador] = useState(0);
5
6    return (
7      <div>
8        <p>Valor actual: {contador}</p>
9        <button onClick={() => setContador(contador + 1)}>Sumar</button>
10     </div>
11   );
12 }
13
14 export default Contador;
15
```

Explicación: `useState(0)` crea la variable de estado `contador` con valor inicial 0 y la función `setContador` para cambiarla. Cada vez que se presiona el botón, se llama a `setContador` con el nuevo valor, React vuelve a renderizar el componente y en pantalla se ve el número actualizado.

¿Qué son los eventos en React?

Los eventos son las acciones que realiza el usuario sobre la interfaz (hacer clic, escribir, enviar un formulario, pasar el mouse). React permite detectarlos y ejecutar una función en respuesta. Sirven para que la aplicación sea interactiva.

¿Cuáles son sus características y para qué sirve?

- ☆ Se escriben en camelCase: onClick, onChange, onSubmit.
- ☆ Se les pasa una función entre llaves, no un texto: onClick={manejarClick}.
- ☆ La función que responde al evento se llama *manejador (handler)*.
- ☆ Reciben un objeto event con información del evento (por ejemplo, event.target.value en un input).
- ☆ Para evitar el comportamiento por defecto (como el envío de un formulario) se usa event.preventDefault().
- ☆ Los más comunes son onClick, onChange, onSubmit, onMouseOver y onKeyDown.

Ejemplo de implementación:

```
1  import { useState } from "react";
2
3  function Saludo() {
4    const [nombre, setNombre] = useState("");
5
6    function manejarCambio(event) {
7      setNombre(event.target.value);
8    }
9
10   function manejarClick() {
11     alert("Hola, " + nombre);
12   }
13
14   return (
15     <div>
16       <input type="text" placeholder="Tu nombre" onChange={manejarCambio} />
17       <button onClick={manejarClick}>Saludar</button>
18     </div>
19   );
20 }
21
22 export default Saludo;
23
```

Explicación: el evento **onChange** del input ejecuta manejarCambio cada vez que se escribe, y guarda el texto en el estado nombre. Al hacer clic en el botón, el evento **onClick** ejecuta manejarClick, que muestra una alerta con el nombre ingresado.

¿Qué son los Hooks en React?

Los Hooks son funciones especiales de React (sus nombres empiezan con use) que permiten usar estado y otras características de React dentro de componentes de función, sin necesidad de escribir clases. Sirven para manejar estado, efectos secundarios, contexto y más, de forma simple y reutilizable. Se incorporaron en React 16.8.

¿Cuáles son sus características y para qué sirve?

- ☆ Solo se pueden usar en componentes de función o en otros Hooks.
- ☆ Se deben llamar siempre en el nivel superior del componente: no dentro de condicionales, bucles ni funciones anidadas.
- ☆ Su nombre siempre empieza con **use**.
- ☆ Permiten reutilizar lógica entre componentes mediante Hooks personalizados (*custom hooks*).
- ☆ Hacen el código más corto y ordenado que con componentes de clase.

¿Cuáles son los principales Hooks?

- ☆ **useState:** permite agregar estado a un componente.
- ☆ **useEffect:** ejecuta código cuando el componente se monta, se actualiza o se desmonta (por ejemplo, pedir datos a una API).
- ☆ **useContext:** permite acceder a datos compartidos entre componentes sin pasarlos por props uno a uno.
- ☆ **useRef:** guarda una referencia a un elemento del DOM o un valor que persiste sin provocar un nuevo renderizado.
- ☆ **useReducer:** maneja estados más complejos mediante funciones reductoras.
- ☆ **useMemo y useCallback:** optimizan el rendimiento evitando cálculos o funciones innecesarias en cada renderizado.

Ejemplo de implementación:

```
1  import { useState, useEffect } from "react";
2
3  function ReLoj() {
4    const [hora, setHora] = useState(new Date().toLocaleTimeString());
5
6    useEffect(() => {
7      const intervalo = setInterval(() => {
8        | setHora(new Date().toLocaleTimeString());
9      }, 1000);
10
11     return () => clearInterval(intervalo);
12   }, []);
13
14   return <h2>Hora actual: {hora}</h2>;
15 }
16
17 export default ReLoj;
18
```

Explicación: **useState** guarda la hora actual en el estado hora. **useEffect** se ejecuta una sola vez al montar el componente (por el arreglo vacío []) y crea un intervalo que actualiza la hora cada segundo. La función que devuelve limpia el intervalo cuando el componente se desmonta, para evitar que siga corriendo en segundo plano.